



COMPUTER ORGANIZATION AND ARCHITECTURE

CSE 2203

LECTURE 8

CHAPTER 7: MICROPROGRAMMED CONTROL

University of Liberal Arts Bangladesh

MICROPROGRAMMED CONTROL

2

- **Control Memory**
- **Sequencing Microinstructions**
- **Microprogram Example**
- **Design of Control Unit**
- **Microinstruction Format**

Ref.: COA by M Morris Manno

Control Unit

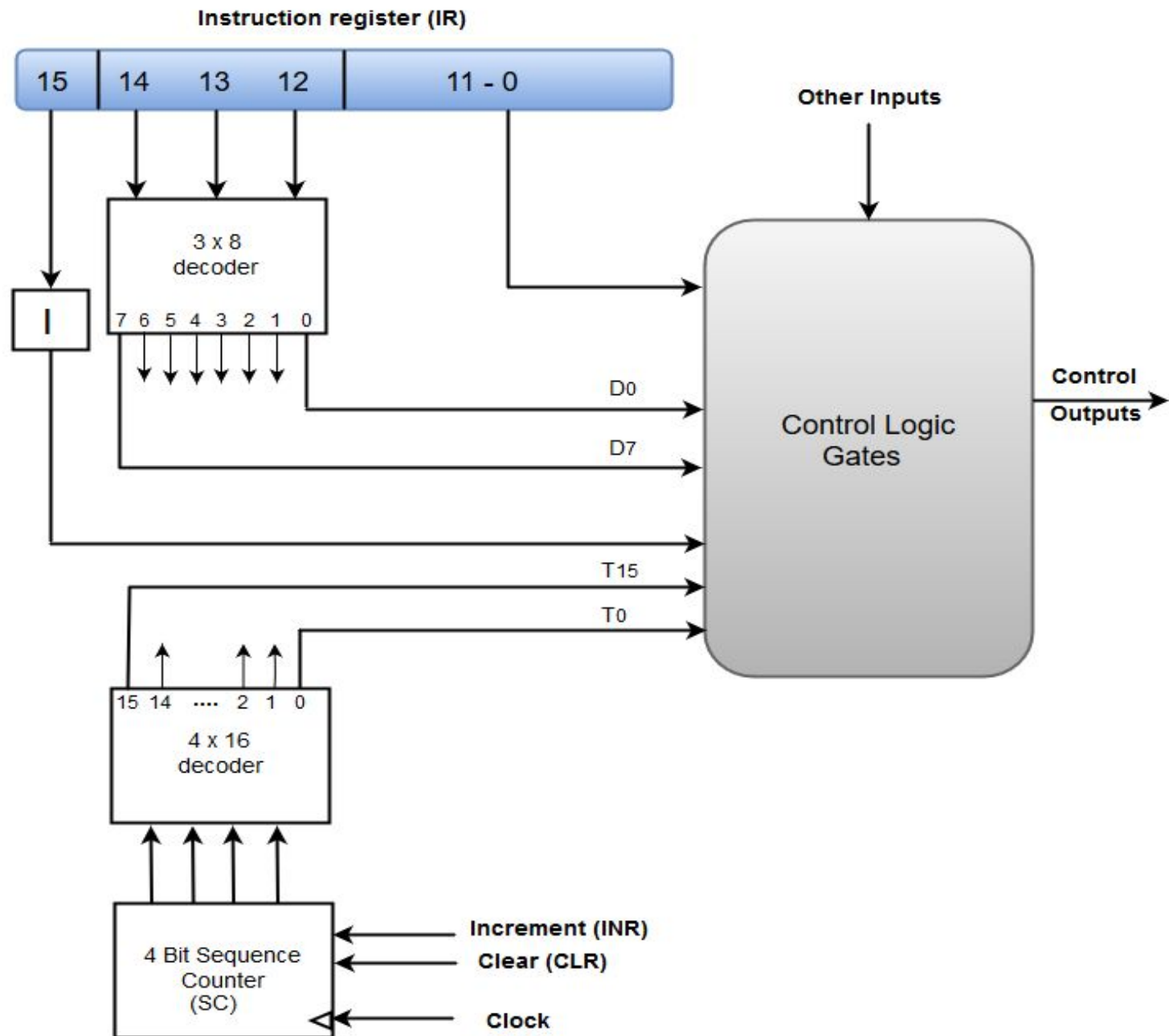
In computer architecture, the control unit is responsible for directing the flow of data and instructions within the CPU. There are two main approaches to implementing a control unit:

- ❖ hardwired and
- ❖ micro-programmed.

Hardwired control unit

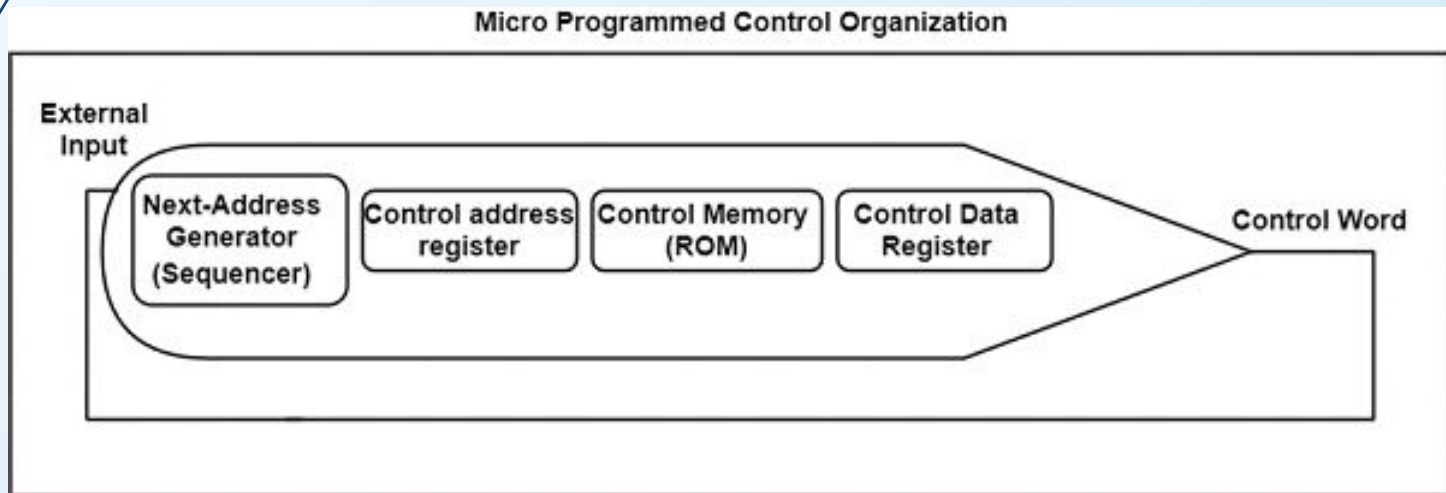
- ❖ A hardwired control unit is a control unit that uses a fixed set of logic gates and circuits to execute instructions.
- ❖ The control signals for each instruction are hardwired into the control unit, so the control unit has a dedicated circuit for each possible instruction.
- ❖ Hardwired control units are simple and fast, but they can be inflexible and difficult to modify.
- ❖ it is used in SPO

Control Unit of a Basic Computer:



Micro-programmed control unit

- ❖ Micro-programmed control unit is a control unit that uses a microcode to execute instructions.
- ❖ The microcode is a set of instructions that can be modified or updated, allowing for greater flexibility and ease of modification.
- ❖ The control signals for each instruction are generated by a microprogram that is stored in memory, rather than being hardwired into the control unit.
- ❖ used in GRO



TERMINOLOGY

7

- ❑ Microprogram
 - Program stored in memory that generates all the control signals required to execute the instruction set correctly
 - Consists of microinstructions
- ❑ Microinstruction
 - Contains a control word and a sequencing word
 - Control Word - All the control information required for one clock cycle
 - Sequencing Word - Information needed to decide the next microinstruction address
- ❑ Control Memory(Control Storage: CS)
 - Storage in the microprogrammed control unit to store the microprogram

❑ *Sequencer (Microprogram Sequencer)*

A Microprogram Control Unit that determines the Microinstruction Address to be executed in the next clock cycle

- In-line Sequencing
- Branch
- Conditional Branch
- Subroutine
- Loop
- Instruction OP-code mapping

MICROINSTRUCTION SEQUENCING

9

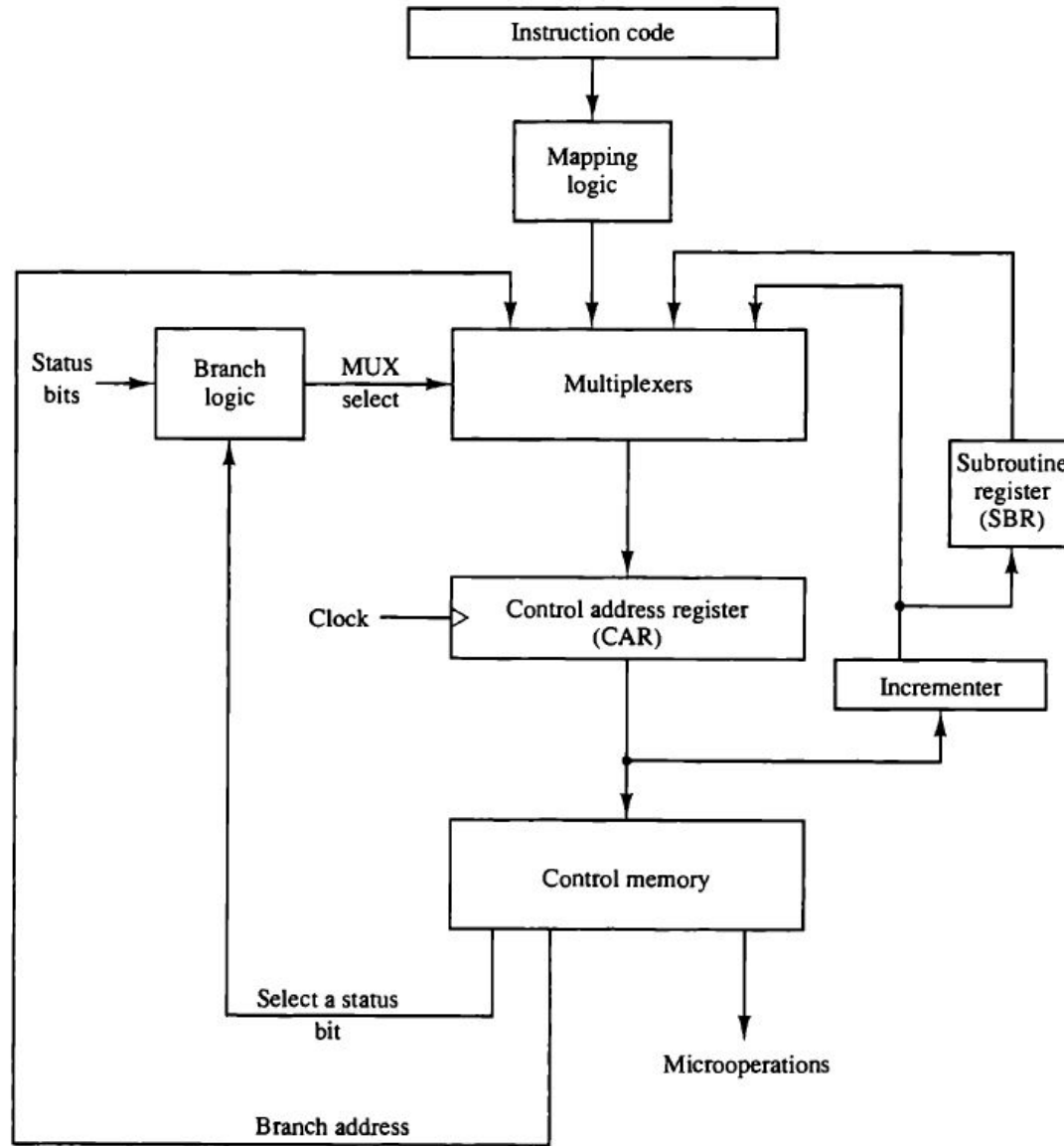


Figure 7-2 Selection of address for control memory.

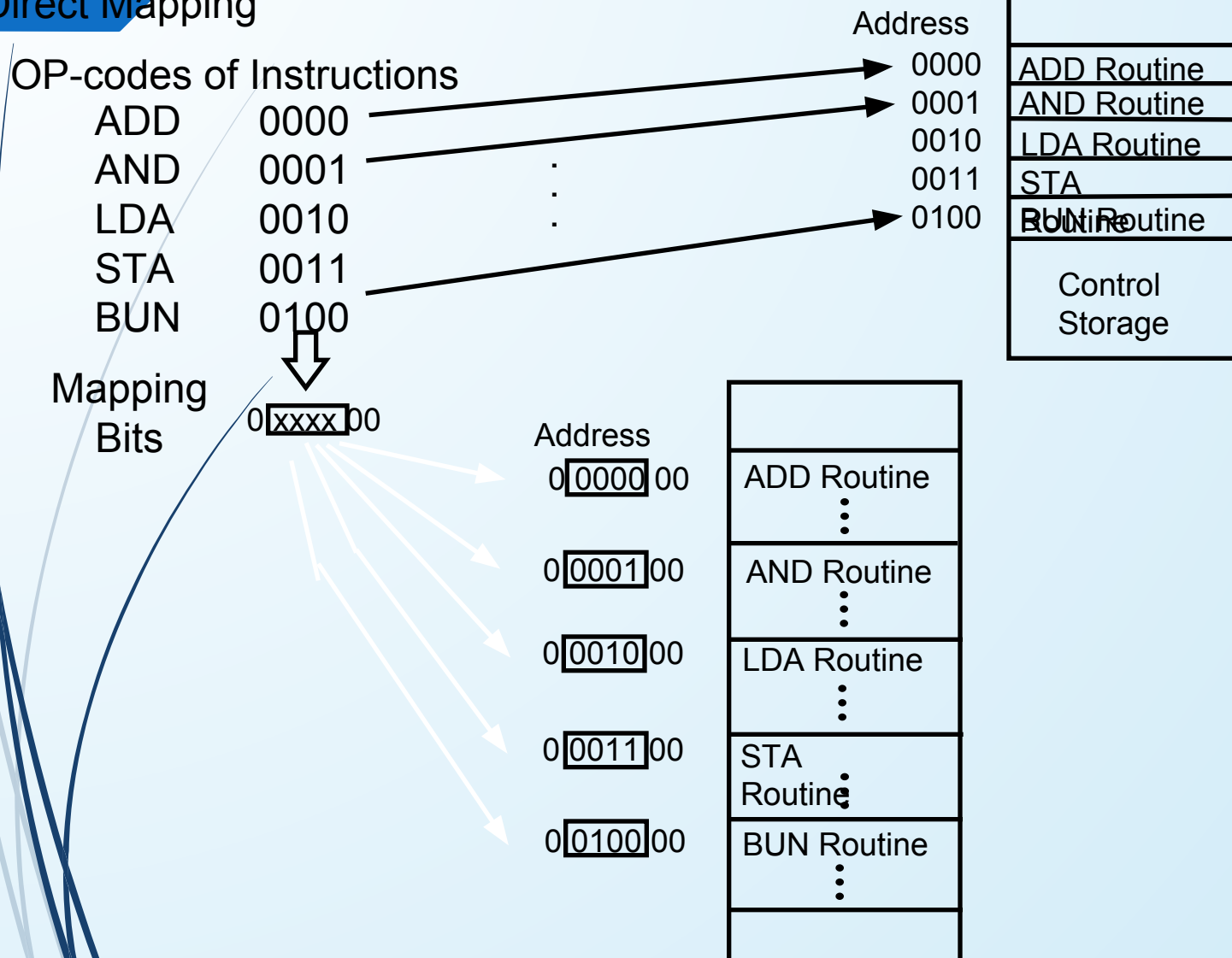
MICROINSTRUCTION SEQUENCING

10 Sequencing Capabilities Required in a Control Storage

The microinstruction in control memory contains a set of bits to initiate microoperations in computer registers and other bits to specify the method by which the next address is obtained. The diagram shows four different paths from which the control address register (CAR) receives the address. The incrementer increments the content of the control address register by one, to select the next microinstruction in sequence. Branching is achieved by specifying the branch address in one of the fields of the microinstruction. Conditional branching is obtained by using part of the microinstruction to select a specific status bit in order to determine its condition. An external address is transferred into control memory via a mapping logic circuit. The return address for a subroutine is stored in a special register whose value is then used when the micro-program wishes to return from the subroutine.

MAPPING OF INSTRUCTIONS

Direct Mapping



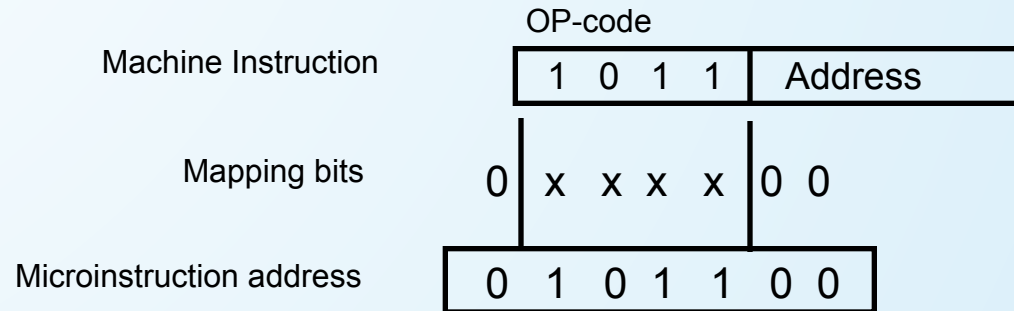
The above image shows a type of easy mapping process which are going to convert the 4-bit operation code into the 7-bit address for control memory. In the mapping process, the 0 will be placed in the most significant bit of address. After that, the four operation code bits will be transferred. Lastly, the two least significant bits of CAR will be cleared.

With the help of this process, a micro-program will be provided to each computer instruction. The micro-program contains the capacity of four microinstructions. If less than four microinstructions are used by the routine, the location of unused memory can be used for other routines. If more than four microinstructions are used by the routine, it will use the addresses 1000000 through 1111111.

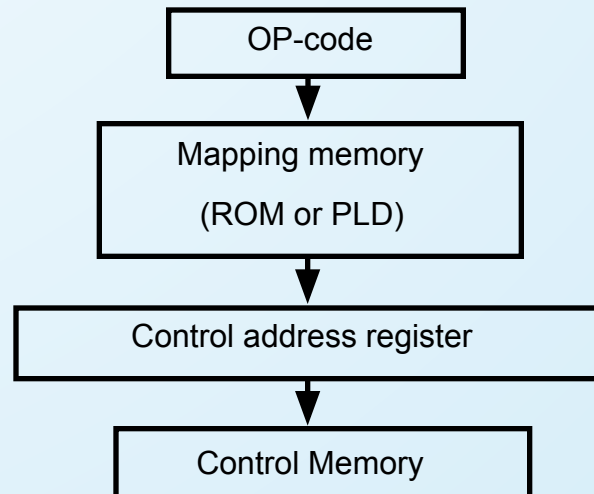
MAPPING OF INSTRUCTIONS TO MICROROUTINES

13

Mapping from the OP-code of an instruction to the address of the Microinstruction which is the starting microinstruction of its execution microprogram



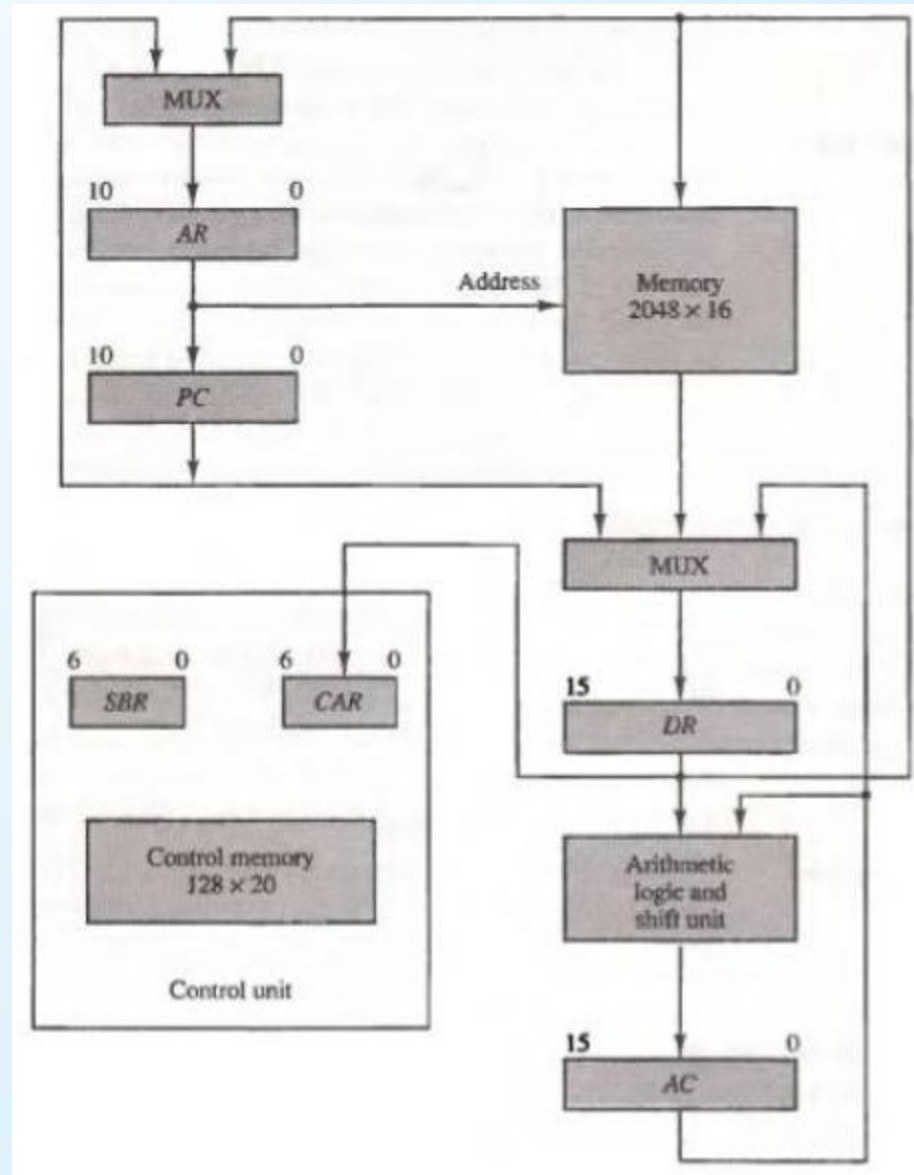
Mapping function implemented by ROM or PLD



MICROPROGRAM EXAMPLE

14

Computer Configuration



MACHINE INSTRUCTION FORMAT

15

Machine instruction format

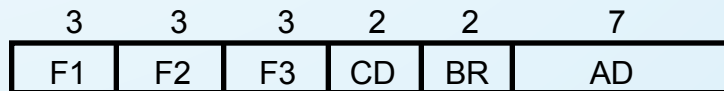


Sample machine instructions

Symbol	OP-code	Description
ADD	0000	$AC \leftarrow AC + M[EA]$
BRANCH	0001	if $(AC < 0)$ then $(PC \leftarrow EA)$
STORE	0010	$M[EA] \leftarrow AC$
EXCHANGE	0011	$AC \leftarrow M[EA], M[EA] \leftarrow AC$

EA is the effective address

Microinstruction Format



F1, F2, F3: Microoperation fields
CD: Condition for branching
BR: Branch field
AD: Address field

MICROINSTRUCTION FIELD DESCRIPTIONS - F1,F2,F3

F1	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC + DR$	ADD
010	$AC \leftarrow 0$	CLRAC
011	$AC \leftarrow AC + 1$	INCAC
100	$AC \leftarrow DR$	DRTAC
101	$AR \leftarrow DR(0-10)$	DRTAR
110	$AR \leftarrow PC$	PCTAR
111	$M[AR] \leftarrow DR$	WRITE

F2	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC - DR$	SUB
010	$AC \leftarrow AC \vee DR$	OR
011	$AC \leftarrow AC \wedge DR$	AND
100	$DR \leftarrow M[AR]$	READ
101	$DR \leftarrow AC$	ACTDR
110	$DR \leftarrow DR + 1$	INCDR
111	$DR(0-10) \leftarrow PC$	PCTDR

F3	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC \oplus DR$	XOR
010	$AC \leftarrow \overline{AC}$	COM
011	$AC \leftarrow \text{shl } AC$	SHL
100	$AC \leftarrow \text{shr } AC$	SHR
101	$PC \leftarrow PC + 1$	INCPC
110	$PC \leftarrow AR$	ARTPC
111	Reserved	

CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
01	$DR(15)$	I	Indirect address bit
10	$AC(15)$	S	Sign bit of AC
11	$AC = 0$	Z	Zero value in AC

BR	Symbol	Function
00	JMP	$CAR \leftarrow AD$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
01	CALL	$CAR \leftarrow AD$, $SBR \leftarrow CAR + 1$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
10	RET	$CAR \leftarrow SBR$ (Return from subroutine)
11	MAP	$CAR(2-5) \leftarrow DR(11-14)$, $CAR(0,1,6) \leftarrow 0$

SYMBOLIC MICROINSTRUCTIONS

17

- Symbols are used in microinstructions as in assembly language
- A symbolic microprogram can be translated into its binary equivalent by a microprogram assembler.

Sample Format

five fields: label; micro-ops; CD; BR; AD

Label: may be empty or may specify a symbolic address terminated with a colon

Micro-ops: consists of one, two, or three symbols separated by commas

CD: one of {U, I, S, Z}, where

- U: Unconditional Branch
- I: Indirect address bit
- S: Sign of AC
- Z: Zero value in AC

BR: one of {JMP, CALL, RET, MAP}

AD: one of {Symbolic address, NEXT, empty}

SYMBOLIC MICROPROGRAM - FETCH ROUTINE

18

During FETCH, Read an instruction from memory and decode the instruction and update PC

Sequence of microoperations in the fetch cycle:

AR $\leftarrow$ PC
DR $\leftarrow$ M[AR], PC $\leftarrow$ PC + 1
AR $\leftarrow$ DR(0-10), CAR(2-5) $\leftarrow$ DR(11-14), CAR(0,1,6) $\leftarrow$ 0

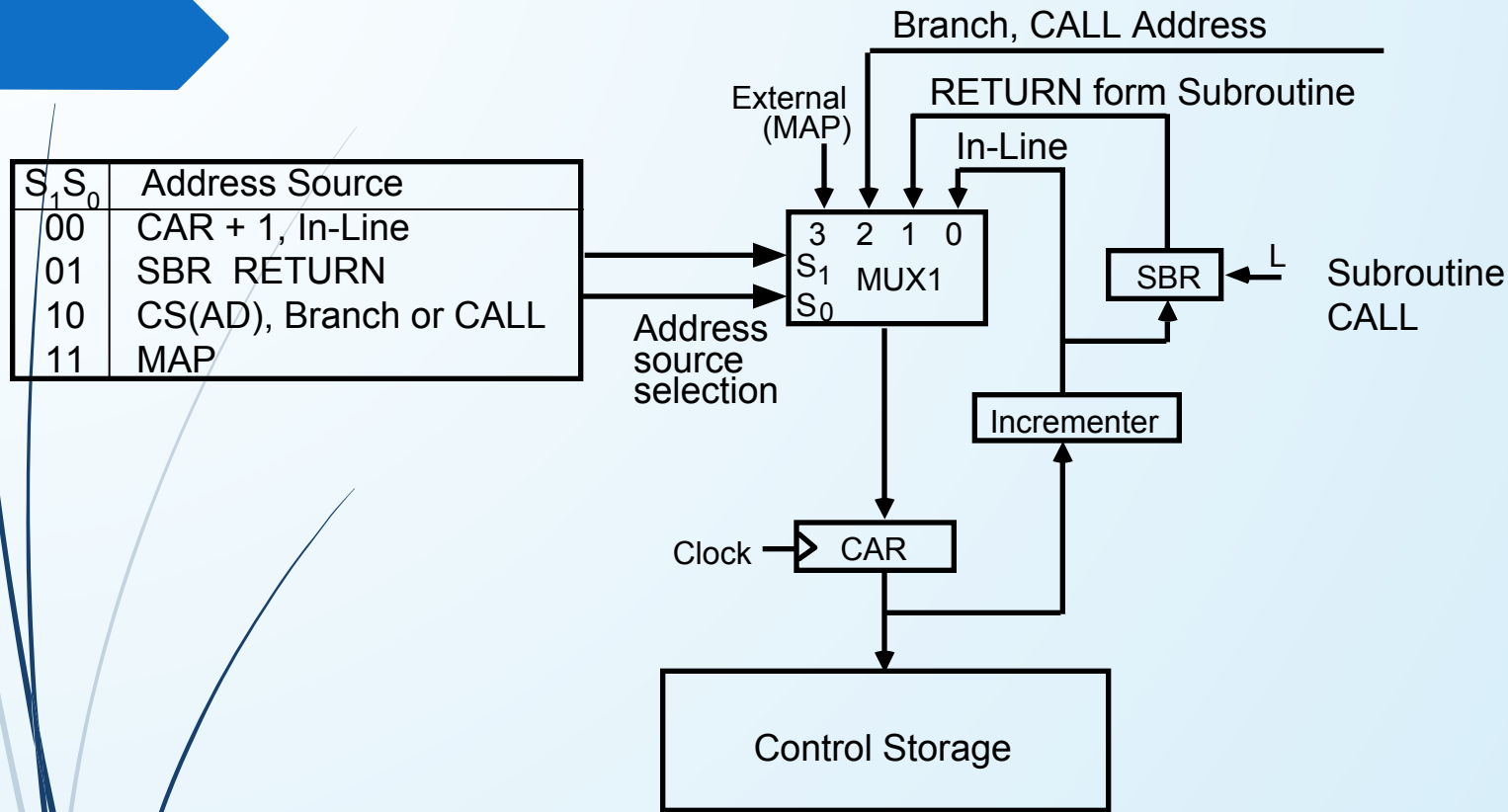
Symbolic microprogram for the fetch cycle:

```
ORG 64
FETCH:  PCTAR      U  JMP  NEXT
        READ, INCPC U  JMP  NEXT
        DRTAR      U  MAP
```

Binary equivalents translated by an assembler

Binary address	F1	F2	F3	CD	BR	AD
1000000	110	000	000	00	00	1000001
1000001	000	100	101	00	00	1000010
1000010	101	000	000	00	11	0000000

MICROPROGRAM SEQUENCER



MUX-1 selects an address from one of four sources and routes it into a CAR

- In-Line Sequencing $\rightarrow$ CAR + 1
- Branch, Subroutine Call $\rightarrow$ CS(AD)
- Return from Subroutine $\rightarrow$ Output of SBR
- New Machine instruction $\rightarrow$ MAP

20

$$L = I_1 I_0 T$$

BR Field	Input $I_1 I_0 T$	MUX 1 $S_1 S_0$	Load SBR L
0 0	0 0 0	0 0	0
0 0	0 0 1	0 1	0
0 1	0 1 0	0 0	0
0 1	0 1 1	0 1	1
1 0	1 0 $\times$	1 0	0
1 1	1 1 $\times$	1 1	0



Ref.: COA by M Morris Manno

Chapter 7

7.3

7.5

7.6

7.7

7.11

7.12



THANK YOU